

Functional Specification for Caching for LLM project

Author	Leen Kabaha, Hala Atamna, Asmaa Agbaria
Version	V0.1
Created	11-4-2026
Last Updated	13-4-2026

1 Overview

This project aims to design and implement a semantic caching system for Large Language Model (LLM) queries.

Modern LLM-based applications provide powerful capabilities for question answering and text generation, but they suffer from high computational cost and latency, especially when similar or repeated queries are processed multiple times.

The goal of this project is to introduce an intelligent caching layer that stores and reuses responses for semantically similar queries, rather than relying on exact string matching. By leveraging embedding models and vector similarity search, the system can detect when a new query is “close enough” to a previously answered query and return a cached response instead of invoking the LLM again.

This project is a new system design and not a direct extension of an existing implementation.

The system is designed to be modular and extensible, allowing integration with different embedding models, vector stores, and LLM providers through interchangeable components.

1.1 Detailed Definitions

- **Prompt** – The input query provided by the user to the system.
- **Response (Reply)** – The output returned to the user, either generated by the LLM or retrieved from the cache.
- **Large Language Model (LLM)** – An external service that generates natural language responses based on a given prompt. The system communicates with it via a remote API.
- **Embedding** – A numerical vector representation of a text prompt, capturing its semantic meaning.
- **Embedding Model** – A model used to convert text prompts into embedding vectors. This component is designed to be interchangeable (pluggable).
- **Vector Cache (Vector Store)** – A storage component that maintains pairs of <embedding, response> and supports similarity-based search.
- **Similarity Metric** – A function used to measure the similarity between two embedding vectors (e.g., cosine similarity or Euclidean distance).
- **Similarity Threshold** – A configurable parameter that determines whether two prompts are considered similar enough to produce a cache hit.
- **Cache Hit** – Occurs when a stored embedding is sufficiently similar to the current query, allowing the system to return a cached response.
- **Cache Miss** – Occurs when no similar embedding is found, requiring the system to query the LLM.
- **Cache Admission Policy** – Defines whether a new <embedding, response> pair should be added to the cache (default: always admit).
- **Cache Eviction Policy** – Determines which entries are removed when the cache reaches its capacity (e.g., LRU or LFU).
- **Cache Orchestrator** – The central component responsible for managing the flow between embedding generation, cache lookup, and LLM querying.

1.2 Operating platforms

The system will be implemented as a backend server-based application composed of multiple independent services that communicate via REST APIs. The architecture is platform-independent and supports both local development and cloud-based deployment.

Development Environment

During development, the system will run locally on a single machine, where all components operate as separate processes. The implementation will use Python as the programming language and FastAPI for building RESTful APIs. Communication between components will be performed over HTTP using JSON format, with optional use of Docker for containerization.

Production / Deployment Environment

In a production environment, the system can be deployed as distributed services in the cloud. Each component can run independently in its own container or instance on platforms such as AWS, GCP, or Azure. Communication between services will be performed over HTTP/HTTPS, enabling scalability, flexibility, and independent management of system components

1.3 Interfaces

The system interfaces are based on **REST APIs over HTTP**, where all components communicate by sending and receiving **JSON** messages.

Internal interfaces include communication between the Cache Orchestrator and other components such as the Embedding Service, Vector Cache, and LLM Connector through defined API endpoints.

External interfaces include integration with remote **Embedding Model APIs** and **LLM APIs**, which are accessed via standard HTTP requests.

The system is designed to be modular, allowing these external services to be replaced without affecting the overall architecture.

2 Features

2.1 Main features

The system will implement a semantic caching layer for LLM-based queries. The main confirmed features include:

- Receiving user prompts through a REST API and returning responses.
- Generating embedding vectors for input prompts using an embedding model.
- Performing similarity search in a vector cache to detect semantically similar queries.
- Returning cached responses when a similarity threshold condition is satisfied (cache hit).
- Forwarding the prompt to an external LLM service when no suitable cache entry is found (cache miss).
- Storing new <embedding, response> pairs in the cache after a cache miss.
- Supporting a configurable cache size with an eviction policy (e.g., LRU or LFU).
- Running each component (orchestrator, embedding service, cache, LLM connector) as a separate process.
- Providing a modular design that allows replacing the embedding model, vector store, and LLM service.

2.2 Secondary features

The following features are planned but may depend on time and external constraints:

- Support for multiple similarity metrics.
- Configurable similarity threshold as a runtime parameter.
- Basic monitoring and logging (cache hit rate, number of LLM calls).
- Support for switching between different embedding models or LLM providers.
- Improved cache policies beyond the default (hybrid admission strategies).

The implementation of these features depends on factors such as available development time, complexity of integration with external APIs, and successful completion of the core system components.

2.3 Nice-to-have features

- A full graphical web interface for end users.
- Advanced distributed deployment with orchestration tools (e.g., Kubernetes).
- Asynchronous processing with message queues for high scalability.
- Advanced evaluation tools for measuring semantic accuracy of cache hits.
- Fine-tuned or custom-trained embedding models.

3 Dependencies

- **Programming Language and Frameworks**

The system is implemented in **Python**, using **FastAPI** for building REST APIs and handling communication between components.

- **HTTP Communication Libraries**

The project uses standard Python libraries such as `requests` or `httpx` for sending HTTP requests between internal services and for interacting with external APIs.

- **Embedding Model Service**

The system depends on an external or local embedding model to generate vector representations of input prompts. The availability and performance of this service directly affect the system's functionality.

- **LLM API / External Q&A Service**

The system relies on a remote LLM API to generate responses in case of cache misses. The system's correctness and performance depend on the availability, latency, and limits of this external service.

- **Vector Storage / Vector Database**

The project depends on a vector storage solution (such as FAISS, Chroma, or an in-memory implementation) to store embeddings and perform similarity search efficiently.

- **Containerization Tools**

Tools such as **Docker** may be used to run different components as separate services and simplify deployment.

4 Assumptions

The system is based on the following assumptions and constraints:

- External embedding and LLM APIs are available and accessible, as the system depends on them to generate embeddings and responses.
- A stable internet connection is required to enable communication with external services.
- The same embedding model is used consistently to ensure compatibility between stored vectors.
- Required libraries (e.g., FastAPI, HTTP clients, vector storage tools) are available for implementation.
- The system runs on a machine capable of handling multiple services or processes.
- The cache size is limited, requiring the use of an eviction policy.
- The project is constrained by development time, which may limit the implementation of advanced features.

5 Related Skills Evaluation

5.1 Previous Experience

- **Python Programming** – Strong experience in writing structured programs and implementing algorithms through coursework and projects.
- **Operating Systems** – Good understanding of processes, memory, and system-level concepts relevant to building multi-component systems.
- **Computer Networks** – Basic knowledge of networking concepts (e.g., HTTP, client-server model), which supports understanding of system communication.
- **Data Structures and Algorithms** – Good understanding of fundamental concepts such as arrays, trees, and search techniques.
- **Version Control (Git/GitHub)** – Practical experience in using Git for code management, collaboration, and version tracking.
- **Software Design** – Basic experience in modular system design and separation of components

5.2 Learning “Theory”

- **Embedding Models** – Understanding how text is represented as vectors and how semantic meaning is captured.
- **Vector Similarity Search** – Concepts behind nearest-neighbor search and similarity metrics such as cosine similarity.
- **Client–Server Architecture** – Understanding how distributed systems handle communication between components.

5.3 Learning Frameworks

- **FastAPI Framework** – Limited experience with building backend services using FastAPI for REST APIs.
- **Vector Databases (e.g., FAISS, Chroma)** – No prior hands-on experience with storing and querying embeddings efficiently.
- **Embedding and LLM APIs** – Limited experience in integrating and working with external APIs for embeddings and language models.
- **Docker (Containerization)** – Basic or no experience in packaging and running services using containers.
- **Multi-service System Integration** – Limited experience in connecting multiple independent services into a single working system.